

Ethan Patterson
Nico Cabrera

HW4 Q3

a.) This is a modification on the traditional knapsack algorithm with an additional knapsack. Instead of having two choices(to put an item in the knapsack or skip it), we have 3(sack 1, sack 2, or skip). For the traditional problem, we look at an individual item, then consider every possible knapsack weight from 1 to n where n is the total weight, and repeat for every item, building off the previous. As we have a second knapsack, we need to repeat this process for the second sack for every single weight increment in the first sack, turning the original 2d array into a 3d one. Despite being nigh impossible to conceptualize mentally, it makes pretty good sense when written out in the code.

The first thing to do is pad the outer(now 3d) layer of the array object with zeroes. The original algorithm does this as well, but this variant of the algorithm adds a dummy object to the start of the array, with a special check for this object so we know when and where to pad. After this, from the top, we iterate over every item, starting from the first (real) item. For each item, we then iterate over every possible increment of weight in the first knapsack. For each increment of weight, we then iterate over every possible increment of weight in the second knapsack. Inside the third nested for loop here, we perform the actual logic behind the solution. It is mostly intact from the original, with some minor modifications. We perform the same set of checks as the original, ensuring that the object can actually fit into the sack, and that the item beats the previous item's value at the same weight(both weights, in this case). One nuance though, we need to determine if an object goes into sack 1 or sack 2. To accomplish this, we calculate the value for both sacks(assuming the object doesn't get rejected for not fitting) and the object goes to the sack that it would produce a higher value in, with a tiebreaker towards sack 1(gotta pick one). If an object does not fit for either or the value of adding it wouldn't be worthwhile, we go to the skip case, and the previous item's values for the two knapsack capacity values are used. After iterating over all the values, the total can be found in the last spot in the array, located at `[itemCount][capacity 1][capacity 2]`.

Backtracking is then performed to derive the items from our completed dynamic programming array, working from the position of the total. We check if the previous item at the capacity positions had a different(guaranteed to be lower) value than the current one. If so, that means the current item was added. Thus, we can subtract the item's value from the total value to find the value before it was added. Then we need to determine if it went to sack 1 or sack 2. We know the value at the max weight for each knapsack. If we subtract the item's weight from the total capacity, one of the two knapsacks should have the value we found before at the position of that weight for the relevant sack. If both have it, it really doesn't matter as that means there's (probably) more than one optimal solution. After this, we have the optimal max total value and the exact objects for each knapsack.

B.) incredibly truncated pseudocode because the array logic is quite a mess

```

//items pre-loaded with dummy object at position 0
knapsack(items, capacity1, capacity2):
    bagdata[items.length][capacity1][capacity2]
    For item in items:
        For w1( 0 up to and including capacity1)
            For w2( 0 up to and including capacity2)
                If Item == items[0]:
                    Bagdata[item][w1][w2] = 0
                    Continue
                Verify bag 1 can hold the item without going over
                Verify bag 2 can hold the item without going over
                Bag1value = item.value + previous item's bag 1 value when bag
1's capacity is exactly enough to fit the new item
                Bag2value = item.value + previous item's bag 2 value when bag
2's capacity is exactly enough to fit the new item
                If bag1Value > bag2Value and can fit in bag 1 and beats out
previous item's bag 1 value at same capacity values for bag 1/bag 2:
                    bagData[item][w1][w2] = bag1Value
                Else if bag2Value > bag1Value and can fit in bag 2 and beats out
previous item's bag 2 value at same capacity values for bag 1/bag 2:
                    bagData[item][w1][w2] = bag2Value
                Else:
                    bagData[item][w1][w2] = bagData[item-1][w1][w2]
    Currentitem = items.length - 1
    Totalbagvalue = bagData[currentitem][capacity1][capacity2]
    Print totalbagvalue
    While totalbagValue > 0:
        If totalBagValue does not equal the previous bag's capacity at currentItem - 1 and
the same capacity spots:
            Confirm item could fit in bag1
            If item fits in bag 1 and previous item at current capacities equals
totalbagvalue- currentitem.value
                Item went in bag 1, append to a bag 1 array for output
            Else item went in bag 2, append to a bag 2 array for output
        Else item got skipped
        Go to previous item

    Adjust output arrays so theyre in id order(currently, theyre highest id to lowest id)
    Print bag1 output array
    Print bag2 output array

```

c.) This is correct because the core algorithm of the knapsack problem is still, fundamentally intact. The main addition is that of a second knapsack that simply adds an additional dimension to the problem and forces it to work through another series of problems and fill out more data in

the dynamic programming array compared to before. Nothing about the fundamental “heart of the solution” changes by adding this dimension. The more optimal(read: more “valuable”) knapsack is always taken between the two at every given choice in the problem, which as long as that is the case, the solution will always be correct, just that we have to check more directions for the dynamic programming memorization to work as intended.

d.) $O(\text{numItems} * \text{capacity1} * \text{capacity2})$

e.) The triple nested for loops are the big o here. Outer loop iterates over items, middle loop iterates over the capacity of sack 1, inner loop iterates over the capacity of sack 2. The backtracking iterates at most n times, in the case that all items fit, which is still a smaller class of function compared to the triple loop.